Lecture 2 · Part I · Lecture notes

The end-to-end project

Dataset: California housing

How to use these notes

The primary text for this lecture is Géron, Chapter 2, with §4.1 brought forward. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

---

1 Where we are

Lecture 1 looked at the data and split it. Five findings carry into today, and three of them say the same thing.

What we found	Why it matters today
One categorical column, `ocean_proximity`, one level with $n = 5$	it has to be encoded, and that level will be absent from some folds
207 missing values in `total_bedrooms`	something must fill them, and that something is *learned from data*
Features on wildly different scales	they have to be scaled, and scaling is *learned from data*
A target capped at \$500,000	4.8% of rows carry a label that is not the answer
16,512 training / 4,128 test, stratified on income	the test rows are sealed until the last fifteen minutes

Three of those five say *learned from data*, and anything learned from data can be learned from the wrong data. That is the subject of §3.

2 Derivation: least squares and the normal equation

You called `LinearRegression().fit()`. What did it compute, and when does that computation fail? The second half of the question is not idle: its answer is exactly the warning that ended Lecture 1 about engineering features as weighted sums of existing ones.

2.1 The model

$$\hat{y} \;=\; \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

with $\hat{y}$ the prediction, n the number of features, x_i the i-th feature and θ_j the j-th parameter. θ_0 is the **bias** or **intercept**. Writing $\mathbf{x} = (x_0, \ldots, x_n)$ with $x_0 \equiv 1$ folds the bias into the dot product:

$$\hat{y} \;=\; h_{\boldsymbol{\theta}}(\mathbf{x}) \;=\; \boldsymbol{\theta} \cdot \mathbf{x} \;=\; \boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}.$$

2.2 What "fit" means

Training means setting $\boldsymbol{\theta}$ so the model best fits the training set, which requires a measure of how badly it fits:

$$\mathrm{MSE}(\mathbf{X}, y, h_{\boldsymbol{\theta}}) \;=\; \frac{1}{m}\sum_{i=1}^{m}\left(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)} - y^{(i)}\right)^2.$$

We minimise MSE rather than RMSE because $\sqrt{\cdot}$ is strictly increasing on $[0, \infty)$, so

$$\arg\min_{\boldsymbol{\theta}} \sqrt{f(\boldsymbol{\theta})} \;=\; \arg\min_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) :$$

same minimiser, simpler algebra. This is the first instance of a pattern worth naming now, because it recurs for the rest of the course: *learning algorithms routinely optimise a different function during training than the one used to evaluate the result* — because it is easier to optimise, or because it carries terms needed only during training.

2.3 The normal equation

Stack the instances as rows of $\mathbf{X}$ and the labels into y:

$$\mathrm{MSE}(\boldsymbol{\theta}) \;=\; \frac{1}{m}\,\|\mathbf{X}\boldsymbol{\theta} - y\|_2^2 .$$

We are minimising a squared Euclidean norm. That is a *geometric* statement, and it is the whole content of this section.

Differentiate, and check the second order

$$\nabla_{\boldsymbol{\theta}}\,\mathrm{MSE}(\boldsymbol{\theta}) \;=\; \frac{2}{m}\,\mathbf{X}^{\intercal}(\mathbf{X}\boldsymbol{\theta} - y)\,,$$

so at a stationary point $\hat{\boldsymbol{\theta}}$,

$$\mathbf{X}^{\intercal}\left(\mathbf{X}\hat{\boldsymbol{\theta}} - y\right) \;=\; \mathbf{0}.$$

Is it a minimum? The Hessian is $\nabla^2_{\boldsymbol{\theta}}\,\mathrm{MSE} = \frac{2}{m}\,\mathbf{X}^{\intercal}\mathbf{X}$, and for any $\mathbf{v}$,

$$\mathbf{v}^{\intercal}\mathbf{X}^{\intercal}\mathbf{X}\mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 \geq 0,$$

so the Hessian is positive semi-definite and the MSE is convex. A convex function has no local minima — only a global one — so any stationary point is *the* minimum. Rearranging gives the **normal equation**:

$$\hat{\boldsymbol{\theta}} \;=\; (\mathbf{X}^{\intercal}\mathbf{X})^{-1}\,\mathbf{X}^{\intercal}y.$$

Convexity is not a technicality to be waved through. It is also the reason gradient descent, in Lecture 4, is guaranteed to reach the same answer this closed form gives.

2.4 What the stationarity condition says geometrically

Read $\mathbf{X}^{\intercal}(\mathbf{X}\hat{\boldsymbol{\theta}} - y) = \mathbf{0}$ again: every column of $\mathbf{X}$ is orthogonal to the residual $\mathbf{r} = \mathbf{X}\hat{\boldsymbol{\theta}} - y$.

As $\boldsymbol{\theta}$ varies, $\mathbf{X}\boldsymbol{\theta}$ ranges over the column space of $\mathbf{X}$. Minimising $\|\mathbf{X}\boldsymbol{\theta} - y\|_2$ therefore finds the point of that subspace closest to y — and the closest point of a subspace to a vector is its orthogonal projection.

The sentence to remember

Fitting a linear model is projecting the label vector onto the span of the features. The residual is what the features cannot reach.

2.5 Computed on our own data

Eight numeric features, imputed and scaled, with a dummy column added, so $\hat{\boldsymbol{\theta}}$ has nine entries. Solving the normal equation directly and comparing with the library:

```
>>> theta_best[:3].round(1)
array([206333.5, -85371.4, -90829.7])
>>> lin_reg = LinearRegression().fit(X, y)
>>> np.abs(theta_best - np.r_[lin_reg.intercept_, lin_reg.coef_]).max()
6.4e-10
```

Identical to floating-point precision. Look at θ_0: \$206,334 — and the mean price of the training set is \$206,334. With centred features the intercept *is* the mean of the target, so the constant predictor we measure later in this lecture is exactly this model with all eight weights set to zero. Least squares spends those eight weights to get from \$115,311 down to \$68,233.

Verify the geometry, on the same data

If the projection argument is right, the residual must be orthogonal to every column of $\mathbf{X}$, so $\mathbf{X}^\intercal\mathbf{r}$ should be numerically zero. It comes out at 7.1×10^{-6} — against $|\mathbf{X}^\intercal y|_{\max} = 3.4 \times 10^{9}$, which is the scale to judge it by. The relative size is 2×10^{-15}: machine epsilon on a double.

Two lessons in one check

The first is that this is a cheap, decisive test that your solver did what the mathematics says. The second is more general: *"small" is meaningless until you divide by something*. 7.1×10^{-6} is enormous if the natural scale is 10^{-9} and invisible if it is 10^{9}.

2.6 When it breaks

The equation requires $(\mathbf{X}^\intercal\mathbf{X})^{-1}$ to exist. $\mathbf{X}^\intercal\mathbf{X}$ is $(n{+}1) \times (n{+}1)$ and is invertible if and only if it has full rank — equivalently, if and only if $\mathbf{X}$ has linearly independent columns. There are exactly two ways to lose that.

Failure condition — more features than instances

If $m < n$ then $\operatorname{rank}(\mathbf{X}) \le m < n + 1$, so $\mathbf{X}^\intercal\mathbf{X}$ is singular. Note what the problem is: not that no parameter vector fits the data, but that infinitely many fit it *exactly*. Common in genomics, in text, and in any wide-feature setting.

Failure condition — collinear features

If one column is a linear combination of the others, the columns are dependent and $\mathbf{X}^\intercal\mathbf{X}$ is singular again.
This is Lecture 1's warning, now explained. "When creating new combined features, avoid simple weighted sums of existing features" — because a weighted sum of existing columns *is* a linear combination, and you would be manufacturing the singularity yourself.

Exact collinearity is rare; near-collinearity is common, and is also a problem. The matrix is invertible but ill-conditioned: tiny changes in the training set produce large changes in $\hat{\boldsymbol{\theta}}$, and the model is numerically unstable. Lecture 5 repairs this with ridge regression, which adds $\alpha\mathbf{I}$ and makes the matrix invertible for *every* $\alpha > 0$ — a fact that is immediate once you have this derivation.

2.7 What Scikit-Learn actually does

It does not invert anything. It computes $\hat{\boldsymbol{\theta}} = \mathbf{X}^{+}y$, where $\mathbf{X}^{+}$ is the Moore–Penrose pseudoinverse, obtained from the singular value decomposition:

$$\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\intercal \qquad \Longrightarrow \qquad \mathbf{X}^{+} = \mathbf{V}\boldsymbol{\Sigma}^{+}\mathbf{U}^\intercal.$$

To build $\boldsymbol{\Sigma}^{+}$: take $\boldsymbol{\Sigma}$, set to zero every value below a small threshold, replace the rest by their reciprocals, and transpose.

The pseudoinverse is always defined, even when the inverse is not. But notice what the threshold is doing. When the inverse does not exist there are infinitely many $\boldsymbol{\theta}$ that fit equally well; zeroing the tiny singular values selects one of them — the minimum-norm least-squares solution, the shortest vector among all the best fits.

A choice, not a discovery

That selection is a convention, not a fact about the data. It is why two rank-deficient fits can agree on every prediction and disagree on every coefficient, and why interpreting coefficients from a collinear design is unsafe even when the predictions are fine.

Method	**In n (features)**	**In m (instances)**
Normal equation	$O(n^{2.4})$ to $O(n^3)$	$O(m)$
SVD (Scikit-Learn)	$O(n^2)$	$O(m)$

Both are linear in the number of instances, so both handle large training sets well provided the data fits in memory. Both become very slow as n grows — say 100,000 features. That is what gradient descent is for.

3 Measuring generalisation

3.1 A number that means nothing

Fit an unconstrained decision tree on the 16,512 training districts and score it on those same districts: RMSE 0.0. No error at all. There are two explanations and only one is flattering.

The model is perfect. Housing prices would have to be an exact deterministic function of nine census attributes, with two identical districts always carrying identical median prices, no measurement noise anywhere in a census, and no influence from anything outside our nine columns. Reject.

The model memorised. An unconstrained tree keeps splitting until every leaf is pure, and with enough depth it isolates every training district into its own leaf. Predicting the training set is then a lookup, and a lookup has zero error.

Such a model is called **nonparametric** — not because it has no parameters, but because their number is not fixed before training, so the structure is free to follow the data. That freedom is exactly what lets it memorise.

Failure condition — a score computed on the rows it was fitted to

Any score computed on the rows a model was fitted to measures how well it reproduces what it has already seen. For a model flexible enough to memorise, that is not evidence about anything else.

Linear regression on the same rows gives $68,233, which happens to be honest to within $49 — but only because the model is too rigid to memorise, and *nothing in the number tells you which case you are in*.

3.2 "Then use the test set"

No — not yet. We are still choosing between models and tuning them. Every time a choice is made by looking at the test score, that score stops being an estimate of performance on unseen data and becomes a target we are fitting to. The test set is used *once*, at the end, to estimate the generalisation error of a system already committed to.

This is the same argument as Lecture 1's specimen exam question, where a loop chose α by scoring on `X_test`. We answer the rest of it in §6.4.

3.3 Leakage: the same error, one step earlier

A score can be corrupted before any model is fitted.

```
scaler = StandardScaler()
housing_scaled = scaler.fit_transform(housing_full)   # all 20,640 rows
train_set, test_set = train_test_split(housing_scaled)  # then split
```

Definition 3.1 (Data leakage)**.** *A quantity computed from rows outside the training set enters the training path.*

Here the mean and standard deviation were computed from a set that includes the test rows, so the model is evaluated on rows whose own values helped define the transformation applied to them. It raises no exception, and the reported score improves.

What the leak cost here — measured

How much it improves is not knowable from the code. It is measurable afterwards. Same data, same models, twenty seeds, scaled before the split against scaled after it; all figures are dollars of RMSE.

Model	seed 42	mean, 20 seeds	sd
Linear regression	0.88	0.02	0.31
Random forest	57.17	3.89	47.87
PCA(4), then linear	15.90	1.85	16.34

Against a split-to-split spread of $1,391 on the same RMSE, every one of those is nothing — and the sign flips from seed to seed. Report the seed-42 column alone and you have published noise.

Why this is an argument *for* the rule

The cost here was nothing. The cost elsewhere is unbounded. And you cannot tell which case you are in by reading the code — which is precisely why the rule has to be procedural rather than a judgement made case by case.

Lecture 1 predicted this: all three models here are scale-invariant or nearly so, and a standardisation is an invertible affine map. The leak was real and its effect was zero, for a reason we could state in advance.

The rule, stated once

Fit every transformer on the *training set only*. Never call `fit()` or `fit_transform()` on the validation set, the test set, or new data. Once fitted, you may `transform()` anything.

This applies to imputers, scalers, encoders, dimensionality reduction — everything that learns anything from data. We will not rely on remembering it; the rest of this lecture builds an object that makes violating it structurally impossible.

3.4 What we need, then

Requirement	Because
An estimate of error on unseen rows	a training score cannot see overfitting
. . . without touching the test set	it is spent the first time it influences a choice
. . . with a *spread*, not one number	two estimates differ only if the spread says so
. . . and all preprocessing refitted inside it	otherwise the estimate leaks and is optimistic

All four are satisfied by one procedure.

4 Cross-validation, pipelines and tuning

4.1 From holdout to k-fold

Hold out a validation set from the training set: train candidates on the reduced set, select on the validation set, retrain the winner on the full training set, evaluate once on the test set. The difficulty is sizing it. Too small and the evaluations are imprecise, so you may select a poor model by chance; too large and the remaining training set is much smaller than the one the final model will use — like selecting a sprinter by watching a marathon.

The resolution is many small validation sets, used in rotation. Split the training set into k non-overlapping **folds**, then train and evaluate k times, holding out a different fold each time:

$$\text{score} \;=\; \frac{1}{k}\sum_{j=1}^{k} \text{RMSE}(\text{model trained without fold } j,\ \text{fold } j)\,.$$

The cost is k fits. The return is not only a mean but a *spread*, which a single holdout cannot give at any price.

Two details in the call that are not decoration

```
cv = KFold(n_splits=10, shuffle=True, random_state=42)
tree_rmses = -cross_val_score(tree_reg, X_train, y_train,
                 scoring="neg_root_mean_squared_error", cv=cv)
```

The minus sign: Scikit-Learn's cross-validation expects a utility (greater is better), so the scorer is the *negative* RMSE and the sign is flipped back.
The explicit `KFold`: passing `cv=10` takes ten contiguous blocks in row order. Two people whose frames are sorted differently then get different folds and cannot see why their numbers disagree.

4.2 The tree, measured honestly

Statistic over 10 folds	RMSE
mean	68,573.73
min	65,499.05
median	69,070.93
max	70,958.42

The honest number for that same tree is $68,574, not zero. And the folds run from $65,499 to $70,958, so any comparison turning on less than a couple of thousand dollars is not a comparison.

4.3 All three models, honestly

Model	Training RMSE	Cross-validated RMSE
Predict a constant	$115,311	$115,300
Linear regression	$68,233	$68,282
Decision tree	$0	$68,574
Random forest	$18,058	$48,687

The gap between the columns is the overfitting, and the first row — which cannot overfit anything — has no gap at all. Reading down:

- **Linear regression:** the columns nearly agree. It is not overfitting; it is *under*fitting.
- **Decision tree:** $0 against $68,574. Severe overfitting — and once measured honestly it is not better than the linear model it appeared to crush.
- **Random forest:** $18,058 against $48,687. Still overfitting, but the best honest score, and 58% below the constant baseline.

4.4 Is $68,574 worse than $68,282?

The tree's mean is $292 above the linear model's and the folds scatter by $2,000. But both models saw the *same* ten folds, so compare them fold by fold and the shared fold-to-fold noise cancels in the difference.

Per-fold difference	Mean	Std	Same sign in all 10?
Decision tree − linear regression	+$292	$1,634	no
Random forest − linear regression	−$19,594	$1,650	yes

The forest is genuinely better. The tree is *indistinguishable* from the linear model: a paired t-test gives $p = 0.59$, and on six of the ten folds the tree is the better of the two.

Failure condition — the habit this is training

Say "the tree is worse" and you have read a ranking out of noise. Two numbers are only different if you have shown that they are — and the paired comparison costs nothing, because you already ran both models on the same folds.

Why does the forest win? It averages many trees fitted on random subsets, and if their errors are not strongly correlated the average is more accurate than any of them. Lecture 7 makes that precise by deriving the variance of an average of correlated predictors.

4.5 Pipelines make leakage structurally impossible

```
full_pipeline = Pipeline([
    ("prep",  preprocessing),                    # the ColumnTransformer
    ("model", RandomForestRegressor(random_state=42)),
])
```

When `cross_val_score` holds out fold j, the *entire* pipeline — imputer, encoder, scaler, model — is fitted on the other folds only. The imputer's median and the scaler's mean never see the held-out fold. Leakage cannot occur by accident, which is a stronger guarantee than any amount of care.

4.6 Tune, instead of guessing

```
param_grid = {"model__max_features": [4, 6, 8, 10, 12],
              "model__n_estimators": [30, 100, 200]}
grid_search = GridSearchCV(full_pipeline, param_grid, cv=cv,
                 scoring="neg_root_mean_squared_error", n_jobs=-1)
```

The double underscore addresses any hyperparameter of any estimator, however deeply nested: `model__max_features` reads as "the step named `model`, its parameter `max_features`".

The winner is `max_features=8`, `n_estimators=200`, scoring $48,180 — improved from $48,687, a gain of $508 against a fold spread of $2,762.

Failure condition — two things to notice, and neither is the improvement

150 fits bought a gain we cannot cleanly distinguish from noise.
And 200 was the *largest* value tried for `n_estimators`. A winner sitting on the edge of the grid means the optimum may lie outside it: we did not find the optimum, we found the edge of what we searched.

Preprocessing has hyperparameters too, and the same path reaches them:

Imputation strategy	Cross-validated RMSE
`median`	$48,180
`mean`	$48,154
`most_frequent`	$48,171

A spread of $26 across the three. With 207 missing values out of 20,640 that is exactly what it should be — and now we know instead of assuming. The reason to put preprocessing inside the pipeline is not that tuning it always helps; it is that preprocessing and model interact, and the pipeline is what lets you find out whether they do *here*.

Randomised search is usually the better spend: it explores as many distinct values as it has iterations rather than the few you listed, adding a hyperparameter that turns out not to matter costs it nothing where a grid multiplies, and — note — it cannot sit on the edge of a grid, because there is no grid. Halving searches go further, running a tournament that evaluates many candidates on a small slice and promotes the survivors with more data each round.

4.7 Analyse the best model

Feature	Importance
`median_income`	0.4418
`ocean_proximity_INLAND`	0.1511
`longitude`	0.1146
`latitude`	0.1043
`housing_median_age`	0.0494
...	
`ocean_proximity_NEAR OCEAN`	0.0062
`ocean_proximity_NEAR BAY`	0.0020
`ocean_proximity_ISLAND`	0.0003

Median income dominates, as the correlation column predicted. Longitude and latitude together take 0.22: the forest has rediscovered the coast from two raw coordinates.

Importance is a reason to *try* dropping a feature and then measure — never a reason to drop it outright, and never a measurement in itself.

The gun loaded in Lecture 1

At the bottom of that list: `ocean_proximity_ISLAND`, at 0.0003. Five districts in California, and the split was stratified on income, not on this.

	Total	In training	In test
`ISLAND` districts	5	2	3

The model has seen the category twice. It will be asked about it three times, and those three answers will be reported inside an average over 4,128.

Failure condition — and in cross-validation it can vanish entirely

`handle_unknown="ignore"` does not raise on a category it never saw. It emits $[0, 0, 0, 0]$ — a district that is neither near the ocean nor inland — silently.
With our seed no fold loses it. Over 500 reshufflings, 11.2% produce at least one fold in which the encoder is fitted without `ISLAND` and then asked about it: a one-in-nine chance of a silently wrong column, invisible in the score. This is what "rare categories will cause trouble" meant.

5 The test set, once

```
final_model = grid_search.best_estimator_   # refitted on all of X_train
final_predictions = final_model.predict(X_test)
final_rmse = root_mean_squared_error(y_test, final_predictions)   # $49,037
```

We `predict` on the test set. We never `fit` anything on it.

5.1 How precise is that estimate?

A bootstrap over the squared errors gives a 95% interval of \$46,752–\$51,379, or roughly ±\$2,313.

Why bootstrap rather than a t-interval? Not the skew — it is 8.9, but with $n = 4{,}128$ the central limit theorem covers the mean anyway. It is that we report the *square root* of the mean, and converting an interval for one into an interval for the other needs the delta method. The bootstrap skips it.

The tuned cross-validated estimate was \$48,180 and the test result is \$49,037: \$858 apart, against an interval of ±\$2,313. They agree. There is nothing here to explain.

Failure condition — the temptation, and why it is fatal

If you tuned a lot, test performance is often slightly worse than the cross-validated estimate, because the system was fine-tuned to the validation data. When that happens you must *not* tweak hyperparameters to improve the test number. Those improvements would not generalise; you would be overfitting the test set, which is the only thing standing between you and a confident fiction.
We searched 15 configurations, so the effect here is invisible. Search 10,000 and it will not be.

5.2 Look at the ten worst predictions

Actual	Predicted	Error	income_cat	ocean_proximity
\$500,001	\$129,715	−\$370,286	2	`<1H OCEAN`
\$500,001	\$130,590	−\$369,412	3	`INLAND`
\$131,300	\$464,117	+\$332,817	5	`<1H OCEAN`
\$500,001	\$171,694	−\$328,307	2	`<1H OCEAN`
\$500,001	\$175,534	−\$324,467	1	`INLAND`

Seven of the ten are capped districts the model under-predicts by a quarter of a million dollars. The third row is the mirror image: a district whose `median_income` is 15.0001 — the *other* cap — and which the model therefore prices as if it were Beverly Hills. It is worth $131,300.

Both caps were spotted in a histogram before any model existed, and they are now the two largest errors of the finished system.

5.3 The average hides the model

One number for 4,128 districts is a summary, not a finding.

income_cat	n	RMSE	ocean_proximity	n	RMSE
1 (poorest)	165	$62,606	`INLAND`	1,250	$39,829
2	1,316	$43,082	`<1H OCEAN`	1,862	$48,967
3	1,447	$48,381	`ISLAND`	3	$54,754
4	728	$53,152	`NEAR OCEAN`	569	$57,191
5 (richest)	472	$54,333	`NEAR BAY`	444	$60,195

The poorest 165 districts are predicted worst in dollars — and their median price is $91,300, so in relative terms it is far worse still. `NEAR BAY` is 51% worse than `INLAND`. And the `ISLAND` number is computed from three districts: report it with the count or do not report it.

A single RMSE of $49,037 describes none of these groups. That is what "check fairness" means operationally, and it is a different report to the client.

5.4 Drop the capped districts?

The cap causes the worst errors, so remove the 205 capped test districts and measure again:

Evaluated on	n	Model RMSE	Constant baseline
All test districts	4,128	$49,037	$115,727
Capped districts removed	3,923	$44,197	$97,908

Failure condition — these two numbers cannot be compared

$44,197 is not an improvement on $49,037. We did not change the model — we changed the *question*. The second row answers "how well does it do on districts below the cap?", which is a different and easier question.
Notice that the baseline fell by 15% as well, which is the tell. Any time a number improves, ask first whether the evaluation set moved. Dropping rows is the cheapest way in the world to improve a metric, and it is almost never a result.

Should we drop them? For *training*, arguably yes — they teach the model a price that does not exist. For *reporting*, only with the sentence above attached.

5.5 The criterion we never measured

The brief said the experts are off by more than 30% *of the price*. We optimised dollars. Measured on the training folds — because comparing two differently-trained models is a choice, and choices are not made on the test set:

Trained on	RMSE	Median % error	Within 30%
the price (what we did)	$48,629	12.0%	85.0%
the log of the price	$49,646	11.5%	86.5%

Regressing the log target is $1,017 worse in dollars and 1.5 points better on the criterion the stakeholder actually stated. Neither is the right answer — but choosing without measuring is not a choice.

Also worth carrying to the client: the cheapest tenth of districts is predicted with an RMSE of $38,441, the dearest tenth with $97,519. In dollars the model is worst where the money is; in percentages it is worst where the money is not.

6 What the number means

Measured how	RMSE	vs the baseline
Predict a constant — the training mean	$115,311	—
Best model, cross-validated on the training set	$48,180	58% better
The same model, on the test set, once	$49,037	58% better

The first row is the one people leave out. Without it, $49,037 is a number with no units of judgement attached to it.

6.1 Is that good?

On the stakeholder's own criterion the system is within 30% on 85.1% of districts, with a median miss of 11.3%. The experts were said to be off by more than 30% typically. So: plausibly better — on an assumption we were handed and never checked.

What a defensible report says

Ask for fifty districts estimated by the expert team and score them on the same test rows with the same metric. Until then the comparison is between a measurement and a claim, and only one of them has an interval.

It may still be worth launching, especially if it frees the experts for work only they can do. But do not deploy it unqualified for the poorest districts, or for the 205 at the cap. "Is the model good?" is not a question about RMSE. It is a question about what the organisation does with it.

6.2 Six questions to ask of any reported number

1. Was every transformer fitted *after* the split, on training data only?
2. Is preprocessing inside the pipeline passed to cross-validation?
3. Was the reported metric computed on held-out data?
4. Was the test set touched more than once?
5. Were hyperparameters selected using validation data, not test data?
6. Does any feature encode information unavailable at prediction time?

Questions 1 and 2 are what Part B of the paper asks when it hands you a code excerpt.

6.3 Delegating the review

An assistant can perform step 3 of Lecture 1's loop, but only if you ask it as an *adversary* rather than an author. "Is this code correct?" invites agreement, and you will usually get it. Instead: *"This notebook reports RMSE $49,037. Assume that number is optimistic. List every path by which test-set information could have reached the training procedure, and cite the line."* Give it the failure to look for and a number to disbelieve.

Bug	**Found from the code alone?**
`fit_transform` on the full set	yes
Preprocessing outside the cross-validated pipeline	yes
Test set evaluated more than once	no — that is session history, not code
A feature unavailable at prediction time	no — that is domain knowledge
Split not stratified on the right variable	no — "right" is your judgement

Three of the five require something no reader of the file can supply. Those three are yours.

6.4 Back to the specimen question

Lecture 1 set a loop choosing α by scoring on `X_test`. Parts 1 and 2 were answerable then; parts 3 and 4 are answerable now.

The repair. Wrap the choice in a grid search over folds carved from the training set, then touch the test set exactly once, after α is fixed. `RidgeCV` does the same job in one line and is equally acceptable. What is not acceptable is any version in which `X_test` appears before the choice is made.

Is the honest number higher? In expectation, yes: the printed value was a minimum over five noisy estimates, and α was chosen to look good on the very set it was scored on. Guaranteed on one split? No — the bias is a statement about repeated draws, not about this sample, and cross-validation may also pick a genuinely better α. Both halves are required; "higher" alone, or "no" alone, is an incomplete answer.

You have now seen the second half measured: our own tuned model came out $858 higher on the test set, well inside the interval — a single split, telling you nothing about the expectation.

7 The notebook

What to change

The notebook repeats the Lecture 1 load and split in its first two cells, so it stands on its own. Run it once end to end, then change the grid's `n_estimators` list to include values above 200 and re-run the search. The winner moved off the edge of the grid — did the score move with it, or by less than the $2,762 fold spread? That one change is the whole argument about grid boundaries.

8 Exercises

Five, in the style of the written paper. Solutions on Lecture 3's deck.

1. The normal equation is $\hat{\theta} = (X^\mathsf{T}X)^{-1}X^\mathsf{T}y$. Your design matrix has a column equal to the sum of two others. State what happens to $X^\mathsf{T}X$, and what `np.linalg.pinv` returns instead. [5]

2. A pipeline scales the features and then fits a ridge model, and is passed whole to `cross_val_score`. Explain what would go wrong if the scaling were done once, before the call. [5]

3. You report a 95% confidence interval for the test RMSE. A colleague says the interval proves the model is better than the baseline. Give the one question you would ask before agreeing. [4]

4. k-fold cross-validation reports a mean of 0.71 with folds $[0.52, 0.79, 0.75, 0.74, 0.75]$. What do you conclude, and what would you do next? [5]

5. Explain, in two sentences, why the test set may be looked at exactly once, and what you may legitimately do if the number disappoints. [4]

Summary

Fitting a linear model is orthogonal projection: the minimiser satisfies $\mathbf{X}^\mathsf{T}(\mathbf{X}\hat{\boldsymbol{\theta}} - y) = \mathbf{0}$, so the residual is orthogonal to every feature, and $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}y$ whenever the inverse exists. It fails to exist when $m < n$ or when the features are collinear — which is why Lecture 1 warned against building features as weighted sums of existing ones. The SVD pseudoinverse is always defined, is cheaper, and silently chooses one of infinitely many fits when the problem is degenerate.

A score computed on the training rows cannot detect overfitting: an unconstrained tree scored 0 and was worth $68,574. The test set is spent the first time it influences a choice. Leakage is a condition, not an accident, and the pipeline makes it structurally impossible rather than merely discouraged — though measured here, over twenty seeds, the leak cost nothing, for a reason Lecture 1 could state in advance.

k-fold gives an honest estimate *and* its precision; two estimates differ only if the paired per-fold difference says so, which demoted the tree from "much worse" to "indistinguishable, $p = 0.59$". Tuning bought $508 against a fold spread of $2,762 and left the winner on the edge of the grid.

The test set was touched once, at $49,037 with a 95% interval of ±$2,313, and then the average was taken apart: by income, by ocean proximity, by the worst ten errors, and against a criterion the client stated and we had not measured.